

REMOTE-ASYMMETRY

Remote asymmetry across the fleet's submodules — investigation and remediation

Scope. §§1–9 are the original OC-2 follow-up on `milosvasic.ru`: the reported hazard that its commits are invisible to a fresh clone because origin fetches from one URL and pushes to two others. **§10 extends this document to design-toolkit's GitLab mirror** — the same defect class (an instrument that can only see the remotes it was told about), found one row lower in this document's own §6 table.

Verified. §§1–9 on 2026-08-27, by direct `git ls-remote` against every configured URL and by GitHub REST identity lookup. §10 on 2026-09-02, by `git ls-remote` against a newly-declared `gitlab` remote plus authenticated and **anonymous** provider probes. Every claim in this file was measured, not inferred.

§6's design-toolkit row is CORRECTED by §10 and must not be read on its own.

1. The reported defect

```
origin FETCHES from git@github.com:milos85vasic/
milosvasic.ru.git but its configured push URLs are only
gitflic.ru:milosvasic/milosvasic-net-v-2 and
github.com:milos85vasic/milosvasic.net.v2. A commit lands on
GitFlic and milosvasic.net.v2, while git submodule update --
init clones milosvasic.ru.git, which never receives it.
```

The configuration asymmetry is real. The hazard it was believed to cause is not. The reasoning above assumes milosvasic.net.v2.git and milosvasic.ru.git are two different repositories. They are one.

2. Measured configuration

```
git -C milosvasic.ru remote -v:
```

```
gitflic    git@gitflic.ru:milosvasic/milosvasic-net-v-2.git
(fetch)
gitflic    git@gitflic.ru:milosvasic/milosvasic-net-v-2.git
(push)
github    git@github.com:milos85vasic/milosvasic.net.v2.git
(fetch)
github    git@github.com:milos85vasic/milosvasic.net.v2.git
(push)
origin    git@github.com:milos85vasic/milosvasic.ru.git
(fetch)
origin    git@gitflic.ru:milosvasic/milosvasic-net-v-2.git
(push)
origin    git@github.com:milos85vasic/milosvasic.net.v2.git
(push)
upstream  git@gitflic.ru:milosvasic/milosvasic-net-v-2.git
(fetch)
upstream  git@gitflic.ru:milosvasic/milosvasic-net-v-2.git
(push)
```

```
git config --get-all remote.origin.pushurl:
```

```
git@gitflic.ru:milosvasic/milosvasic-net-v-2.git
git@github.com:milos85vasic/milosvasic.net.v2.git
```

Umbrella's recorded gitlink — `git ls-tree HEAD milosvasic.ru:`

```
160000 commit 66c8d607b20f0d9e984cb0e06f10a2020ebd9a10
milosvasic.ru
```

```
.gitmodules pins the submodule to git@github.com:milos85vasic/
milosvasic.ru.git.
```

3. The three URLs' actual tips

All three were queried directly. **All three are at the identical tip, which is exactly the umbrella's recorded gitlink**, and all three carry a byte-identical tag set (pre-restyle, v0.9.0-baseline, v1.0.0 ... v1.8.0):

URL	refs/heads/main	Holds gitlink 66c8d60?
git@github.com:milos85vasic/ milosvasic.ru.git	66c8d607b20f0d9e984cb0e06f10a2020ebd9a10	yes — it is the tip
git@gitflic.ru:milosvasic/ milosvasic-net-v-2.git	66c8d607b20f0d9e984cb0e06f10a2020ebd9a10	yes — it is the tip
git@github.com:milos85vasic/ milosvasic.net.v2.git	66c8d607b20f0d9e984cb0e06f10a2020ebd9a10	yes — it is the tip

A fresh `git submodule update --init` resolves correctly today. There is no dangling gitlink and no invisible commit.

4. Mirrors or divergent repos? — the decisive finding

The three URLs are not three repositories. They are **two**:

(a) milosvasic.ru.git and milosvasic.net.v2.git are the SAME GitHub repository. The GitHub REST API resolves both names to one object:

```
GET /repos/milos85vasic/milosvasic.ru      -> id 439095641,
node_id R_kgD0GiWRWQ,
                                           full_name
"milos85vasic/milosvasic.ru"
GET /repos/milos85vasic/milosvasic.net.v2  -> id 439095641,
node_id R_kgD0GiWRWQ,
                                           full_name
"milos85vasic/milosvasic.ru"
```

Same numeric id, same node id, same `created_at` (2021-12-16T18:51:56Z), same `pushed_at`, same size (42761 KB), same `ssh_url` (git@github.com:milos85vasic/milosvasic.ru.git). The canonical name that GitHub reports for *both* lookups is `milosvasic.ru`.

The repository was renamed **from** `milosvasic.net.v2` **to** `milosvasic.ru`. `milosvasic.net.v2.git` is a **stale rename-alias**, and GitHub's permanent redirect for renamed repositories is what carries pushes through it. So the fetch URL *does* receive every push — indirectly, via that redirect. `gh repo list milos85vasic` confirms no separate `milosvasic.net.v2` repository exists.

(b) `gitflic.ru:milosvasic/milosvasic-net-v-2` is a genuine, separate **mirror** on a different host, carrying that host's own legacy slug for the same project. It is **in sync** — identical main tip and identical tags. Not divergent.

Conclusion: mirrors, in sync. Nothing has diverged. No history reconciliation is needed or attempted.

5. What the real defect is

Not invisibility — **fragile addressing**. Two smaller, genuine problems remain:

1. **The GitHub push target is named by a stale alias**. Pushes reach the canonical repository only because GitHub still honours the rename redirect. That redirect is not a permanent guarantee: it **breaks the moment anyone creates a new repository at `milos85vasic/milosvasic.net.v2`**. At that point pushes would silently begin landing in the wrong repository while the fetch URL quietly fell behind — producing exactly the failure originally reported, for real. This is not hypothetical namespace pressure: a separate repository `milos85vasic/milosvasic.net` **already exists**.
 2. **The invariant “origin’s fetch URL appears among origin’s push URLs” is violated here and nowhere else** (see §6). Even where it is currently harmless, it defeats inspection: `git remote -v` cannot be read as evidence that pushes reach the cloned URL.
-

6. Same shape in the other submodules?

Checked all seven. `milosvasic.ru` is the only submodule whose origin fetch URL is absent from its own push-URL list.

Submodule	Fetch URL in push set?	Push targets	Mirrors in sync?
<code>milosvasic.ru</code>	NO — only via rename alias	2	yes (66c8d60)
<code>monetization</code>	yes	4	yes — all 4 at 54ed7b0
	yes	6	

Submodule	Fetch URL in push set?	Push targets	Mirrors in sync?
submodules/constitution			yes — all 6 at a09b1ea
ai_interviewing	yes	1	yes
vasic.digital	yes	1	yes
design-toolkit	n/a (no pushurl; pushes to origin.url)	—	yes NO — see §10
submodules/superspec	n/a (no pushurl; third-party)	—	yes

The design-toolkit “Mirrors in sync? yes” above was FALSE when written, and the reason is this document’s own subject matter. It was measured by enumerating the remotes *this checkout declares*, and this checkout declared exactly one for design-toolkit: GitHub origin. A **second** mirror — git@gitlab.com:vasic-digital/design-toolkit.git — existed and was **not** in sync. An instrument that enumerates only the remotes it has been told about reports “in sync” about the remotes it can see and says nothing whatever about the ones it cannot. That is the same blind-instrument class §6 was written to close for milosvasic.ru, surviving one row lower in the same table. Measured and corrected 2026-09-02 in §10.

Every umbrella gitlink was checked for presence on its remote:

- ai_interviewing ed73d855, design-toolkit efd2c3fb, submodules/superspec c20ac6c1, vasic.digital 6e5411c2, monetization 54ed7b0f, milosvasic.ru 66c8d607 — all equal their remote HEAD.
- submodules/constitution gitlink 448981a differs from remote main (a09b1ea), but merge-base --is-ancestor confirms **448981a is an ancestor of a09b1ea** — reachable in a fresh clone. The submodule’s local branch is simply 62 commits behind. Pinning to an older ancestor is a version choice, not a hazard.

No submodule anywhere in this umbrella has an unreachable gitlink.

7. Recommended remediation

The evidence matches the “all mirrors, in sync” branch — but the correct implementation is **replacement, not addition**.

Adding `milosvasic.ru.git` as a *third* pushurl (the remediation proposed for that branch) would be wrong here: it and `milosvasic.net.v2.git` are the same repository, so every push would transmit to it **twice**, and the stale alias would survive as the latent trap described in §5.1.

Recommended — normalize the stale alias to the canonical name.

Same repository, so it cannot redirect work anywhere new; it only removes the dependency on GitHub’s rename redirect and restores the readable invariant:

```
git -C milosvasic.ru config --replace-all remote.origin.pushurl \
    git@github.com:milos85vasic/milosvasic.ru.git
'milosvasic\.net\.v2'
git -C milosvasic.ru remote set-url github
git@github.com:milos85vasic/milosvasic.ru.git
```

This is local, reversible git config. It is strictly de-risking: the push destination is provably unchanged (id 439095641), only the name used to address it changes.

.gitmodules needs no change. It already names the canonical URL `git@github.com:milos85vasic/milosvasic.ru.git`. No umbrella commit is required for the remote fix.

Out of scope by constraint. Nothing is pushed into the submodule. Nothing needs to be: all mirrors already hold the umbrella’s gitlink.

8. Applied

Applied 2026-08-27, exactly as recommended in §7. Post-change state:

```
gitflic    git@gitflic.ru:milosvasic/milosvasic-net-v-2.git
(fetch)
gitflic    git@gitflic.ru:milosvasic/milosvasic-net-v-2.git   (push)
github    git@github.com:milos85vasic/milosvasic.ru.git
(fetch)
github    git@github.com:milos85vasic/milosvasic.ru.git       (push)
origin    git@github.com:milos85vasic/milosvasic.ru.git
(fetch)
origin    git@gitflic.ru:milosvasic/milosvasic-net-v-2.git   (push)
```

```
origin    git@github.com:milos85vasic/milosvasic.ru.git      (push)
upstream  git@gitflic.ru:milosvasic/milosvasic-net-v-2.git
(fetch)
upstream  git@gitflic.ru:milosvasic/milosvasic-net-v-2.git  (push)
```

origin's fetch URL now appears among its push URLs. The GitFlic mirror is untouched and still receives every push.

To revert:

```
git -C milosvasic.ru config --replace-all remote.origin.pushurl \
    git@github.com:milos85vasic/milosvasic.net.v2.git
'milosvasic\.ru'
git -C milosvasic.ru remote set-url github
git@github.com:milos85vasic/milosvasic.net.v2.git
```

Still open for the operator (agent may not do these):

- Nothing requires pushing. All three URLs already hold gitlink 66c8d60.
- The gitflic / upstream remotes are duplicates of one another (same URL, two names). Harmless, but upstream conveys no information here. Left alone.
- The GitFlic slug milosvasic-net-v-2 is that host's own legacy name. Renaming it there is a GitFlic-side operation, not a git-config one, and was not done.

9. Note for OC-2 — ~~not acted on~~ SUPERSEDED 2026-08-27

This section's recommendation is void. It recommends extending an Override §11.4.156 to cover pages.yml. **There is no override to extend.** An override was sought and disqualified twice over: §11.4.156's closing formula names and refuses the exemption vocabulary ("*No escape hatch — no --allow-ci, --enable-workflow, --keep-pipeline, --remote-ci-OK, --ci-exempt flag*"), and the inheritance contract ("*extend them — they do NOT weaken or override any universal clause*") makes a project-local override of an inherited clause structurally impossible. Compliance was therefore the only remaining path that does not amend shared governance, and the operator chose it: **"Comply — disable both, enforce locally."** ~~pages.yml is renamed to a~~

~~non-active .disabled name per §11.4.156(B), alongside the umbrella's ci.yml; the gates move to a local pre-push hook. No constitution amendment was made.~~

Second update, same day — pages.yml was NOT disabled after all. The deploy-vs-test asymmetry argued below is not merely correct; it is *stronger* than this document stated, and it reversed the decision. `gh api repos/milos85vasic/milosvasic.ru/pages` returns `build_type: "workflow"`: GitHub Pages publishes `https://milosvasic.ru/` **exclusively** by running that workflow. There is no `gh-pages` branch, no `docs/` folder, and the repository root is Jekyll SOURCE, so it cannot be served raw from a branch; `_tools/deploy-langs.sh` pushes source and then sleeps waiting for that workflow, so it is **not** a fallback. Disabling `pages.yml` would not make publishing manual — it would end it. Operator's overriding directive, verbatim: ***“Make sure all pages websites work flawlessly! No website can be broken! All websites we have here are running deployed in production!”***

pages.yml is ACTIVE and stays active. Only the umbrella's `ci.yml` was disabled; its gates moved to a local pre-push hook. `milosvasic.ru` is a **known, documented deviation** from §11.4.156 — **still not an override**, and never to be recorded as one. **Honest boundary (§11.4.6):** the umbrella's remaining provider-side surfaces (org-default required workflows, branch-protection required checks, scheduled exports) are operator-only and unverified, so even `ci.yml`'s rename does not by itself prove *“a push triggers ZERO runs”*. Full record: [DECISION-11-4-156-COMPLY.md](#) §0.

`milosvasic.ru/.github/workflows/pages.yml` is an active root workflow in an owned submodule (on: `push [main], workflow_dispatch`; it builds Jekyll and deploys to GitHub Pages). §11.4.156(A) forbids it and (C) scopes the clause to *“repositories we author + push”* — `milosvasic.ru` is authored and pushed here, so the clause applies exactly as it does to OC-1.

Should the Override §11.4.156 recorded for OC-1 extend to this workflow? An override should name it explicitly rather than absorb it silently:

- **The two are not the same case.** OC-1's `ci.yml` is a *test* workflow — the only mechanical check this root has, given INVENTORY gap G5. `pages.yml` is a *deploy* workflow: it publishes `https://milosvasic.ru/`. Disabling it does not weaken enforcement, it takes

a live site offline. That is a stronger practical reason to keep it and a different justification, so it deserves its own sentence in the override rather than inheriting OC-1's reasoning.

- **Scope differs.** An override recorded in the umbrella's `Constitution.md` governs the umbrella. `pages.yml` lives inside a submodule with its own working tree; §102 reserves committing there to the operator. An umbrella override should therefore state explicitly that it covers the owned submodule `milosvasic.ru`, or the file remains formally unresolved.

~~**Recommendation: yes, extend it — but by naming `milosvasic.ru/.github/workflows/pages.yml` and its deploy-not-test justification explicitly in the same override §11.4.156 entry.**~~ A single unqualified override text would leave OC-2 ambiguous.

Withdrawn 2026-08-27 — see the banner at the head of this section. The override never existed, so there was nothing to extend or to name. What the recommendation was actually protecting against — OC-2 being absorbed silently into OC-1's reasoning — is instead honoured by giving `pages.yml` its own row and its own deploy-not-test justification in `DECISION-11-4-156-COMPLY.md` §4, and by the separate treatment in its §0.

Vindicated in substance, though. The bullet above — “*Disabling it does not weaken enforcement, it takes a live site offline*” — was the correct reading, and it understated the case: with `build_type: "workflow"` there is no fallback publisher at all. That argument is what carried the reversal. The mechanism it proposed (an override) was unavailable; the conclusion it reached (`pages.yml` must keep running) is the one that stands, recorded as a **documented deviation** rather than an override.

`Constitution.md` was **not** edited *by this document* — another agent held it at the time. It has since been reconciled with the decision; its OC-1 and OC-2 sections now carry the reversal.

10. design-toolkit's GitLab mirror — measured 2026-09-02

Verdict: 1 — a real finding. The mirror is **6 commits behind** the GitHub origin and is **private while the origin is public**. Both halves are measured, neither is inferred. Nothing was pushed, merged, or re-pinned to produce them.

10.1 Why this needed doing at all

Until 2026-09-02 this checkout declared **no** GitLab remote for design-toolkit — `git -C design-toolkit remote -v` returned origin only. So the mirror's lag and the mirror's visibility were both **unmeasurable from this tree**, and every statement about them in this repository was a *report*, not a measurement. CLAUDE.md said so explicitly, and was right to.

Two consequences followed, and both were live defects:

1. The lag figure in circulation — “**5 commits behind**”, from 2026-09-01 — was measured against GitHub head 5467a888, which is **no longer the head**. A lag stated against a moved reference point does not merely age; it becomes arithmetically wrong in a knowable direction (it can only understate).
2. `scripts/verify-submodule-remote-sync.sh` reported, and still reports, design-toolkit **CURRENT**. That verdict is **true about GitHub** and says nothing at all about the mirror — see §10.6.

Remedy applied: the mirror was added as a named remote. This is local git config only:

```
git -C design-toolkit remote add gitlab git@gitlab.com:vasic-  
digital/design-toolkit.git  
git -C design-toolkit remote -v  
  
gitlab  git@gitlab.com:vasic-digital/design-toolkit.git  (fetch)  
gitlab  git@gitlab.com:vasic-digital/design-toolkit.git  (push)  
origin  git@github.com:vasic-digital/design-toolkit.git   (fetch)  
origin  git@github.com:vasic-digital/design-toolkit.git   (push)
```

This change is NOT tracked and cannot be committed. A submodule's config lives in the superproject's `.git/modules/design-toolkit/config`, which is outside every working tree. A fresh clone therefore has **no** gitlab remote and reproduces the original blindness.

That is a property of git, not an oversight, and it is exactly why the finding is written down here rather than left to live in one machine's config.

10.2 The two heads

```
git -C design-toolkit ls-remote gitlab HEAD
git -C design-toolkit ls-remote origin HEAD
git -C . ls-tree HEAD design-toolkit

520c436c2c2a33ed3976856463347519b3a710d9 HEAD # gitlab
mirror
e7f3815ec35c0940515296ffb3481cd0fab4bfa6 HEAD # github
origin
160000 commit e7f3815ec35c0940515296ffb3481cd0fab4bfa6 design-
toolkit # gitlink
```

The gitlink **equals** the GitHub origin head. It is the *mirror* that is behind, not the pin.

No fetch was required. All three commits were already in the submodule's object store, so ancestry is computed exactly rather than reported as undetermined:

```
git -C design-toolkit cat-file -t
520c436c2c2a33ed3976856463347519b3a710d9 # commit
git -C design-toolkit cat-file -t
e7f3815ec35c0940515296ffb3481cd0fab4bfa6 # commit
```

10.3 Ancestor, not divergent — and the TRUE lag is 6

```
git -C design-toolkit merge-base --is-ancestor 520c436c
e7f3815e ; echo $? # 0
git -C design-toolkit merge-base --is-ancestor e7f3815e
520c436c ; echo $? # 1
git -C design-toolkit rev-list --left-right --count
520c436c...e7f3815e # 0 6
git -C design-toolkit merge-base 520c436c
e7f3815e # 520c436c...
```

The mirror head is a **strict ancestor** of the GitHub head: **0** commits exist on GitLab that GitHub lacks, **6** exist on GitHub that GitLab lacks, and the merge base is the mirror head. This is **lag, not divergence**. No history reconciliation is needed; a push would fast-forward.

TRUE lag = 6 commits, produced by `git -C design-toolkit rev-list --left-right --count 520c436c...e7f3815e → 0 6`.

The superseded “5” is not contradicted, it is **superseded and explained**:

```
git -C design-toolkit rev-list --left-right --count
      520c436c...5467a888      # 0  5
git -C design-toolkit rev-list --count
      5467a888..e7f3815e      # 1
```

5 + 1 = 6. The mirror did not move; the origin did. **Do not restate the 5.**

The six commits the mirror lacks (git -C design-toolkit log --oneline 520c436c..e7f3815e):

```
e7f3815 fix(upstreams): give this repository its own publication
recipes
5467a88 fix($11.4.6): make the mirror-lag figure re-derivable
instead of rotting
7d9240e fix($11.4.6): record the GitLab mirror; absence needs its
own evidence
efd2c3f Add constitution pointer carriers (AGENTS/CLAUDE/QWEN/
GEMINI)
725e456 cascade: feat(002): anti-slop spec – credential-seam
sanitisation + 2 scanner findings
16e4e76 feat(proposed): OpenDesign Learning Kit – reusable,
decoupled component pack
```

Note what the mirror is missing: **the four constitution pointer carriers** (efd2c3f) and both \$11.4.6 fixes about this very mirror. The GitLab copy of design-toolkit predates that submodule’s governance onboarding entirely.

10.4 The visibility asymmetry — CONFIRMED, four ways

The asymmetry is **real and settled**, not a footnote. It was measured authenticated *and* anonymously on both providers, because an authenticated answer alone cannot distinguish “private” from “you happen to have access”.

```
glab api "projects/vasic-digital%2Fdesign-toolkit" | jq
      -r .visibility
gh  api repos/vasic-digital/design-toolkit --jq
      '.visibility, .private'
curl -s -o /dev/null -w '%{http_code}\n' https://gitlab.com/api/
v4/projects/vasic-digital%2Fdesign-toolkit
curl -s -o /dev/null -w '%{http_code}\n' https://api.github.com/
repos/vasic-digital/design-toolkit
```

probe	GitLab mirror	GitHub origin
authenticated API	visibility = private	

probe	GitLab mirror	GitHub origin
		visibility = public, private = false
unauthenticated HTTP status	404	200

glab auth status reported Logged in to gitlab.com as milos85vasic (GITLAB_TOKEN), so the authenticated GitLab answer is a real read of the project, not a guess from a failure.

One repository, two provider identities, two different visibilities.

The GitHub origin vasic-digital/design-toolkit is PUBLIC; the GitLab mirror vasic-digital/design-toolkit is PRIVATE. The 2026-09-01 report that the mirror “is reported to remain private” is now **measured** and correct.

Further measured project facts (glab api projects/vasic-digital%2Fdesign-toolkit):

```
default_branch      = main
last_activity_at    = 2026-08-08T09:05:45.568Z
archived            = False
empty_repo          = False
```

against GitHub’s pushed_at = 2026-09-01T14:14:56Z. The mirror has not received a push in **more than three weeks**; the origin was pushed the previous day.

10.5 Does the PRIVATE mirror hold anything the PUBLIC origin does not?

No — measured, and bounded. This matters for ../content-boundary.md: a private repository is normally a place content must not flow *out* of. Here the flow is the other way, and the private side is a strict subset.

Both remotes advertise a **byte-identical tag set**, and only refs/heads/main differs:

```
git -C design-toolkit ls-remote gitlab
git -C design-toolkit ls-remote origin
```

```

gitlab
origin
HEAD          520c436c...
e7f3815e...
```

```

refs/heads/main      520c436c...
e7f3815e...
refs/tags/v0.2.0     22b6b2db... ^{} 13a76eb2...
identical
refs/tags/v0.2.1     ba55f5f1... ^{} 74a037f5...
identical
refs/tags/v0.2.2     c99c0f5d... ^{} 1ffcd55d...
identical

```

Every commit the mirror advertises — its main tip and all three peeled tags — is reachable from the **public** GitHub head:

```

for c in 13a76eb2 74a037f5 1ffcd55d 520c436c; do
  git -C design-toolkit merge-base --is-ancestor $c e7f3815e &&
    echo "$c ANCESTOR-OF-PUBLIC-HEAD"
done

```

All four print ANCESTOR-OF-PUBLIC-HEAD. The mirror declares exactly one branch (glab api projects/.../repository/branches → ['main']).

The mirror’s **non-git** stores were enumerated too, because a git-ref comparison says nothing about them:

GitLab store	count
merge requests (state=all)	0
issues (state=all)	0
wiki pages	0
snippets	0
container-registry repositories	0
packages	0

Honest boundary (§11.4.6). `git ls-remote` reports only **advertised** refs, and the API counts above are what that token is permitted to see. This is therefore a measurement of the mirror’s *advertised and enumerable* content, not a proof about hidden refs, CI variables, or deploy secrets, which were not probed and are not claimed either way.

10.6 The remote-sync gate CANNOT see the mirror, and adding a remote did not change that

Stated plainly: adding the gitlab remote does not make the mirror visible to `scripts/verify-submodule-remote-sync.sh`. Not partially — not at all.

The gate never enumerates a submodule’s named remotes. It reads one URL per submodule straight out of `.gitmodules` and probes that:

```
# scripts/verify-submodule-remote-sync.sh
while IFS=$'\t' read -r path url; do          # url comes
    from .gitmodules
    ...
    remote_out="$(run_bounded git ls-remote "$url" "$ref_label"
2>&1)"; lsrc=$?
```

`.gitmodules` has **one** url key per submodule, and for `design-toolkit` it is `git@github.com:vasic-digital/design-toolkit.git`. A named remote in the submodule’s own config is never consulted, so the mirror is invisible to the gate by construction. Run *after* the remote was added, the gate confirms it — the `design-toolkit` row names the GitHub URL and nothing else:

```
bash scripts/verify-submodule-remote-sync.sh
```

```
design-toolkit          e7f3815ec35c e7f3815ec35c CURRENT
PASS  REMOTE-SYNC design-toolkit – gitlink e7f3815ec35c... ==
HEAD at git@github.com:vasic-digital/design-toolkit.git
```

That `CURRENT` is true and is not a bluff — it is a complete answer to a narrower question than a reader assumes it answers. It means “the `gitlink` equals the GitHub tip”. A reader looking for “this submodule is in sync everywhere” will misread it, which is precisely how the §6 table above came to record `design-toolkit` as “mirrors in sync: yes”.

The gate documents four honest boundaries in its own header. **This class is not among them.** Boundary 4 says it compares against *one REF per submodule*; the undocumented limit is that it compares against *one REMOTE per submodule*. The same single-URL blindness is shared by `scripts/verify-content-boundary.sh`, which derives visibility from `submodule.<name>.url` — one URL — and therefore records `design-toolkit` as **public** with no way to see that a second identity for the same repository is **private**.

What would be needed — NOT DONE, and deliberately so. Closing this is a change to a registered gate’s contract and is a separate operator decision:

1. **A declared source of mirror URLs.** `.gitmodules` has no schema for a second URL, so the declaration cannot live there. `helix-deps.yaml` is the natural home (e.g. a `mirrors:` list per `deps[]` entry), or a dedicated declaration file. It must be *declared*, never discovered from local config — local config is untracked (§10.1), so a

discovering gate would give different verdicts on different machines, which is the defect it is meant to end.

2. **Per-mirror probing and per-mirror verdicts** in the gate, with an unreachable or unauthenticated mirror mapping to state 2, never folded into CURRENT.
3. **New verdict arithmetic.** Today's summary line counts "of N owned gitlink(s) probed"; with mirrors the denominator becomes remotes, not gitlinks.
4. **A matching paired-mutation proof** under --prove-failure (R5), including a case where origin is current and the mirror is behind — the exact state measured here — which **must** return 1 and not 0.

None of that was done. Changing it alters a check registered in scripts/check-registry.tsv and its proof contract.

10.7 Also measured, and not caused by this work

bash scripts/verify-submodule-remote-sync.sh exits **0** today — *12 CURRENT, 0 DRIFT, 0 UNDETERMINED of 12 owned gitlink(s) probed, 13 declared*. The figure recorded in the root carriers (*exit 1; 6 CURRENT / 6 DRIFT, 2026-09-01*) is a **dated observation that no longer holds**; the gitlinks were bumped by other work between the two runs. Nothing in §10 moved a gitlink. Both runs are correct measurements of different days — which is the reason that gate re-measures instead of caching.

10.8 Standing state after this section

question	answer	how
mirror head	520c436c2c2a33ed3976856463347519b3a710d9	git ls-remote gitlab HEAD
origin head = gitlink	e7f3815ec35c0940515296fffb3481cd0fab4bfa6	git ls-remote origin HEAD; git ls-tree HEAD design-toolkit
relationship	mirror is a strict ancestor — lag, not divergence	merge-base --is-ancestor → 0 / 1 rev-list --left-right --count 520c436c...e7f3815e → 0 6
TRUE lag	6 commits	
mirror visibility	PRIVATE	glab api + anonymous 404

question	answer	how
origin visibility	PUBLIC	gh api + anonymous 200
private-only content	none advertised or enumerable	every mirror ref is an ancestor of the public head; 0 MRs/issues/wiki/snippets/packages
gate sees the mirror?	NO , and adding a remote did not change it	gate probes .gitmodules url, one per submodule

Nothing was pushed. Nothing was fetched (unnecessary — every commit was already local). **No gitlink moved, no merge was performed, nothing was committed.** The only mutation is the untracked gitlab remote in `.git/modules/design-toolkit/config`, reversible with:

```
git -C design-toolkit remote remove gitlab
```

Open for the operator, and only the operator. Pushing main to the mirror (`git -C design-toolkit push gitlab main` — a fast-forward, per §10.3) would close the 6-commit lag. Whether the mirror should be public, private, or retired is likewise an operator decision in a provider UI. Neither was done.

Constraints observed

No push inside any submodule. No `lumen index / purge`, no `codegraph index`. `Constitution.md`, `scripts/`, `.github/workflows/ci.yml`, `.ashlrcode/` and `design-system/` untouched. The `commit wrapper` was deliberately **not** run: it performs `git add .`, which would stage another agent's in-flight `Constitution.md` edits. This document is left uncommitted for the operator.